



one asset, one lien.

A NEUTRAL FIRST TO FILE LIEN REGISTRY ON CREDITCOIN · BUIDL CTC 2026 · RWA

Two lending protocols that never heard of each other will each lend against the same collateral, because neither can see what the other recorded. Singleton witnesses their pledges from the outside, through an inclusion proof re-checked by Creditcoin's BlockProver precompile, asks them for nothing, and refuses the second claim. It is UCC-9 first to file for on-chain collateral, without a trusted operator.

WHY IT CAN ONLY EXIST HERE

L060 to L064 are write only, receipts live in a trie execution never touches, and BLOCKHASH reaches back 256 blocks. So a neutral witness of somebody else's lending is either an off-chain indexer, which is a trusted party and therefore not neutral, or a contract that consumes an inclusion proof of that log. The second exists through BlockProver 0x0FD2 and nowhere else.

HOW ONE PLEDGE BECOMES A RECORD

01 THE PROTOCOL'S OWN LOG

A lender emits its native event. NFTfi and Blur Blend are read on Ethereum mainnet, unmodified and unaware.

02 THE PRECOMPILE

BlockProver verifies the proof inside the accepting transaction, past a depth read from ChainInfo 0x0FD3. The registry checks the source receipt status itself, because the precompile does not.

03 THE REGISTER

First to file is recorded and given a soulbound certificate. Anything second reverts, and the refusal is kept on file for the next lender.

LIVE NOW, CHECKABLE WITHOUT ASKING US FOR ANYTHING

12

inclusion proofs on CC3 testnet, every hash published

2 / 2

attested source chains read, Sepolia and Ethereum mainnet

61

tests, both security reviews kept as regressions

0

wallets, backends or indexers between the page and the chain

THE DEMONSTRATION

Two tokenised deeds, two unrelated Sepolia lenders. On the first, a pledge is recorded, a second refused on chain as a **failed transaction** decoding to **AssetNotFree**, then settled, released and re-filed by the lender that lost the race. On the second, the refusal stays on file because that lien is never released. Six more proofs read real NFTfi and Blur Blend loans on mainnet, one ending in seizure after a failed auction. **Both mainnet protocols escrow**, checked on chain: they prove the reader works and cannot prove the fraud, which is why the collision is ours and why caveat 6 says so.

WHAT IT DOES NOT CLAIM

- **A positive record, not proof of absence.** Attestcoin proves a transaction happened, never that one did not.
- **The adapter is trusted for meaning, not existence.** Eighty lines of pure code, named as semi-trusted before anybody asks.
- **Custody prevents this fraud mechanically**, and the non-custodial market for unique assets is empty. We read that as cause, not objection: lenders take possession because there is no register to check.

ATTESTCOIN PROTOCOL USE, STATED AS CALLS

BlockProver 0x0FD2 in its view form inside the accepting transaction; ChainInfo 0x0FD3 for the attested tip and to resolve a chain key from a chain id; the deployed EvmV1Decoder; @gLuwa/usc-sdk for proof assembly; and both attested source chains, six proofs each. Replay protection is a nullifier per operation domain, keyed by chain, height, transaction index and log index. Every unused surface carries a written reason in docs/SURFACES.md, including the three worth naming: batch verification, the ChainInfo bounds queries, and AttestorStash.

TWO INDEPENDENT REVIEWS, ONE DAY, BOTH FOUND REAL BUGS

The first: a borrower could attach a decoy log carrying the registry's own event signature and make their genuine pledge permanently unregistrable. The second found the fix incomplete, because the emitter was still inferred from a receipt the borrower orders. The registry now searches for nothing: the relay names the emitter and the log index and the registry only checks. A batch pledge became supported rather than refused, and mainnet gas fell from 716k to 634k. Both attacks are kept as regression tests, along with a corrected caveat that had claimed an administrator cannot fabricate, which was false.

ADDRESSES

Registry 0x7F4A466E0bdAD924AaEa8b1f863F477Eb336A950 · CC3 testnet, verified on Blockscout

Read on mainnet NFTfi v3 0xB6adEc2ACc851d30d5fB64f3137234BCDCBBad0D

Blur Blend 0x29469395eAf6f95920E59F858042f0e28D98a20B

Demo lenders Harbor 0xaaD02e7Bebc37AcB5dc67c42F70d61d8C86dF3e5 · Meridian 0xfA72380654232c5538d1F17e2D8d6c261bd263AD